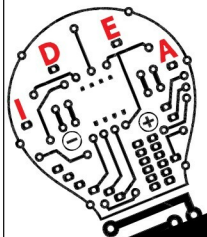


Original, innovative circuit ideas are invited

from designers and
experimenters
for publication in EFY magazine



Honorarium (minimum ₹1,000)
will be paid to the contributors soon
after publication of their articles.

Complete details may be sent to the
Editor, Electronics For You,
D-87/1, Okhla Industrial Area Phase 1,
New Delhi 110020
or emailed to editsec@efy.in

For editorial guidelines, visit
http://efymag.com/Guidelines_for_EFY_authors.doc

DO-IT-YOURSELF

LCD (LCD1) is wired in 4-wire mode with Arduino pins contrast control and backlight enabled.

BC548 transistor (T2) is connected to pin 5 of Arduino for switching on the local alarm LED (LED1) and a buzzer connected across CON4. Another BC548 (T1) is connected to pin 10 for de-energising a relay (RL1) in case the alarm is triggered for industrial PLC interfacing for safety interlocks. Pins 11, 12, 9, 8, 7 and 6 are used for LCD control and data lines. When the setup is powered, and while it is still, it reads



Fig. 3: Initialising mode



Fig. 4: Monitoring mode



Fig. 5: Indicating mode

and stores current accelerometer values in Arduino internal EEPROM regardless of its orientation.

Since the ADC is 10-bit, special header file <EEPROMAnything.h> has been provided with the code. A five-second delay has been provided for all voltages and for the system to be stable before any initial value is read. Arduino's microcontroller reads all three axes data from the accelerometer and stores in the EEPROM. It also stores the default threshold value of 25 in the EEPROM.

Some conventional indications on the LCD (Fig. 3, Fig. 4 and Fig. 5) are shown here for different working modes. In initialising mode (Fig. 3), system parameters are

EFY Note

The source code of this project is included in this month's EFY DVD and is also available for free download at source.efymag.com

initialised. In monitoring mode (Fig. 4), the system enters into monitoring mode with current threshold value displayed on the second line of the LCD.

In indicating mode (Fig. 5), the system reads accelerometer values continuously and compares these with previous steady values of the accelerometer, stored in the EEPROM while initialising. If current value differs, that is, if stored value is either more than threshold value in positive side or less than threshold value in negative side, the alarm sounds and the relay is de-energised. This design and coding supports positive as well as negative values in all three axes.

Pushbuttons connected to pins 2 and 3 of Arduino serve as interrupts for incrementing and decrementing threshold values for sensitivity adjustments. For earthquakes, a threshold of 10 to 15 is good. The sensor can also be used to detect knocks and vibrations if the threshold is set to 8.

The entire setup can be wired and enclosed in a hard enclosure and mounted anywhere in industry or home. Users can also calculate resultant acceleration by using formulae of square root of $X^2 + Y^2 + Z^2$, where X, Y and Z are outputs from ADXL335, and then compare the result with the threshold to raise an alarm. Modifications can be done by the user on the same platform, if required.

Software comprises earthQuake.ino with some header files of Arduino. **EFY**

Ronit Adhiraj Ghosh is M.Tech from Homi Bhabha National Institute, Mumbai. His hobbies include electronics projects, watch collection, camping and hiking